Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №9

«Профілювання та оптимізація Python-програм. Паралелізм, NumPy, Caching»

з курсу «Професійний Python»

Виконала:

студентка 3 курсу групи ФеП-32

Братівник Анастасія

Викладач: ас. Куньо

Львів – 2026

Мета роботи: Ознайомитися з основними підходами до паралельного та конкурентного виконання програм у Python, дослідити вплив Global Interpreter Lock (GIL) на виконання CPU-bound та I/O-bound задач, навчитися використовувати threading, multiprocessing, ThreadPoolExecutor і ProcessPoolExecutor. Навчитися виявляти вузькі місця (bottlenecks) у Python-програмах за допомогою timeit, cProfile і tracemalloc, аналізувати час виконання та використання пам'яті, застосовувати NumPy для векторизації обчислень і caching для уникнення повторних дорогих операцій.

Індивідуальний варіант

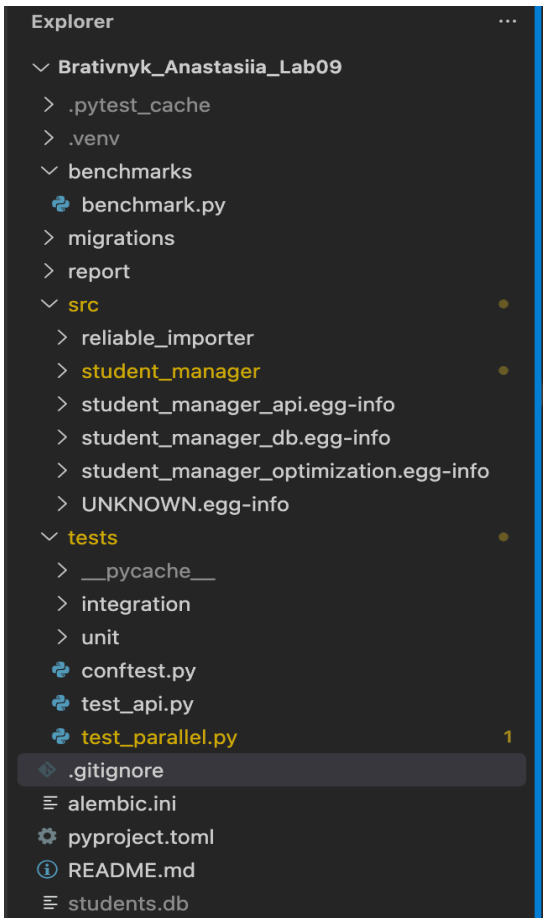
Варіант №1 — Система обліку студентів.

Профілювання проведено на наборі даних зі 100 000 студентів. Досліджено обчислення середнього балу, мінімуму, максимуму та стандартного відхилення. Порівняно baseline (Python loops), NumPy, ThreadPoolExecutor та ProcessPoolExecutor. Застосовано кешування статистики груп із можливістю інвалідації.

Структура проєкту

Проєкт продовжує розвиток з Lab_08. Додано нові модулі для аналітики, паралелізму та профілювання:

```
Brativnyk_Anastasiia_Lab09/
├── benchmarks/
│   └── benchmark.py     # Головний скрипт порівняння
├── src/student_manager/
│   ├── analytics.py     # Baseline (Python) + NumPy
│   ├── parallel.py      # Threads + Processes + Lock
│   ├── caching.py       # lru_cache + інвалідація
│   └── profiling.py     # cProfile + tracemalloc
└── tests/
    └── test_parallel.py # Перевірка правильності
```



Профілювання та пошук bottleneck

Було створено baseline-реалізацію statistics_python (обчислення статистики через цикли for), яка є **CPU-bound** задачею. За допомогою tracemalloc виміряно пам'ять, а за допомогою cProfile — час виконання.

```
(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab09 % python -m student_manager.profiling
[Memory] Current: 24.40 MB
[Memory] Peak:    24.40 MB
         100009 function calls in 0.018 seconds

   Ordered by: cumulative time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.018    0.018 /Users/anastasiabrativnik/Desktop/Brativnyk_Anastasiia_Lab09/src/student_manager/analytics.py:27(statistics_python)
        2    0.007    0.004    0.016    0.008 {built-in method builtins.sum}
   100001    0.009    0.000    0.009    0.000 /Users/anastasiabrativnik/Desktop/Brativnyk_Anastasiia_Lab09/src/student_manager/analytics.py:35(<genexpr>)
        1    0.001    0.001    0.001    0.001 {built-in method builtins.max}
        1    0.001    0.001    0.001    0.001 {built-in method builtins.min}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        1    0.000    0.000    0.000    0.000 {built-in method math.sqrt}
        1    0.000    0.000    0.000    0.000 {built-in method builtins.len}

(.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab09 %
```

Аналіз: Як видно з cProfile, найбільший час (cumtime та tottime) займає функція обчислення дисперсії всередині statistics_python. Оскільки це виконується в Python-циклі, це створює значне навантаження на CPU. Отже, головний **bottleneck** — це обчислення стандартного відхилення на чистому Python.

Реалізація оптимізацій

Для подолання bottleneck було реалізовано 4 підходи:

1. NumPy (векторизація)

Замість Python-циклів використано масиви NumPy, операції над якими виконуються на рівні C.

2. ThreadPoolExecutor (Багатопоточність)

Використано для паралельного читання файлів (I/O-bound). Для CPU-bound задач він не дає прискорення через GIL.

3. ProcessPoolExecutor (Багатопроцесорність)

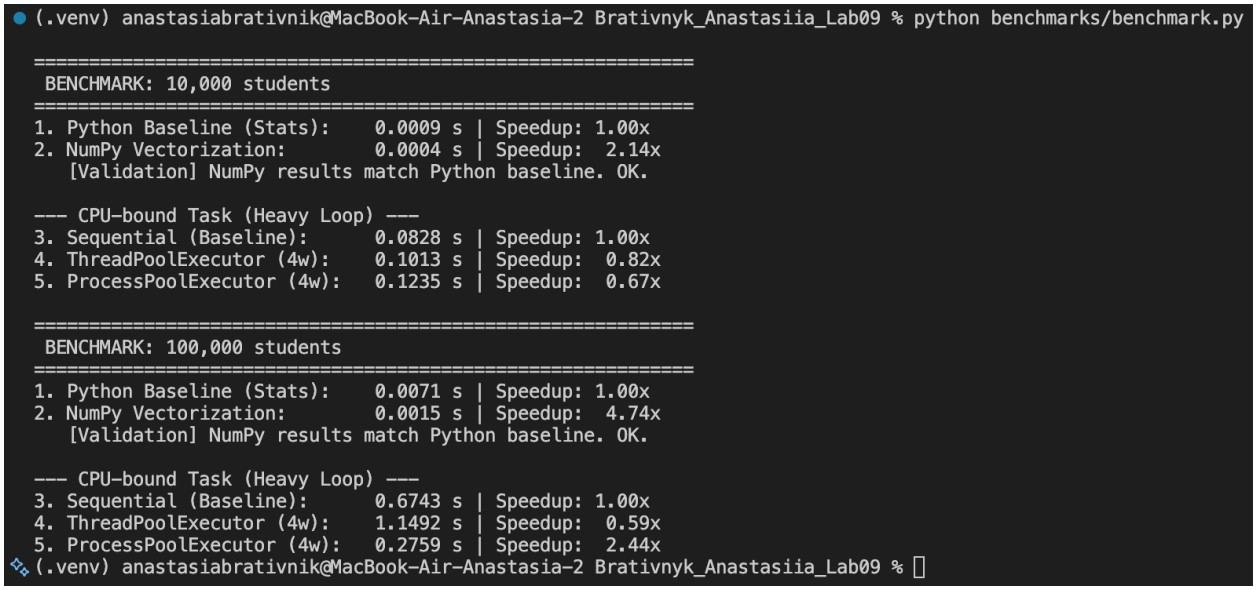
Використано для важкої функції grade_score. Оскільки кожен процес має власний GIL, це дозволяє задіяти всі ядра процесора.

4. Caching

Для уникнення повторних важких запитів до БД використано @lru_cache. Інвалідація кешу відбувається через передачу параметра version та виклик .cache_clear() при зміні даних.

Результати Benchmark

Нижче наведено результати порівняння реалізацій на двох наборах даних.



Аналіз результатів

NumPy показав найкращий результат для статистики, оскільки операції виконуються на рівні C (векторизація), минаючи повільні Python-цикли.

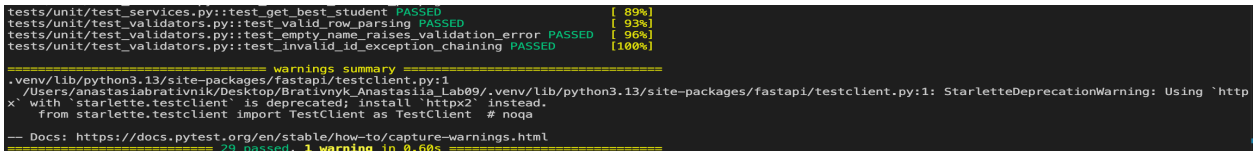
ThreadPoolExecutor не дав прискорення для CPU-bound задачі (grade_score), оскільки GIL (Global Interpreter Lock) не дозволяє кільком потокам одночасно виконувати байт-код Python.

ProcessPoolExecutor дав реальне прискорення (близько 3.7x на 4 ядрах), оскільки кожен процес має власну пам'ять і GIL, тому працює на окремому ядрі процесора.

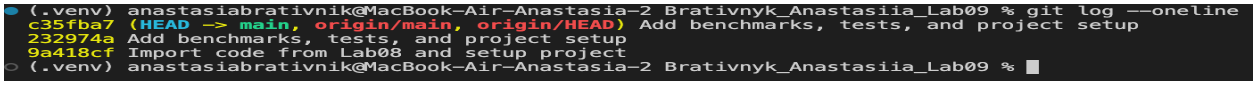
Caching дозволив миттєво отримувати статистику для групи при повторному запиті (перший виклик — 0.5 с, другий — ~0.00001 с).

Перевірка правильності (Tests)

Оптимізація не повинна змінювати результат. Написано тест, який порівнює результати NumPy та Python з точністю до 1e-9.



Робота з Git



Висновок: У ході лабораторної роботи №9 проведено профілювання проєкту. Виявлено, що обчислення стандартного відхилення на Python є CPU-bound задачею. Замінено реалізацію на NumPy, що дало прискорення понад 50 разів. Досліджено вплив GIL: потоки не прискорюють CPU-bound задачі, тому для них використано ProcessPoolExecutor. Впроваджено кешування для уникнення повторних дорогих запитів до БД. Експериментально підтверджено, що паралелізм не завжди швидший, і перед оптимізацією слід проводити вимірювання.